

简易炸弹人

EasyBomber — 基于 C++17 与 EasyX 的单人小游戏

项目名称	EasyBomber（简易炸弹人）
学号	1030425220
姓名	张子昂
提交日期	6 月 21 日
讲解视频网址	https://www.bilibili.com/video/BV1mBjx6cExe/
GitHub 项目网址	https://github.com/TONYCHARLES-52151314/EasyBomber

目 录

CONTENTS

1 项目概述.....	3
1.1 运行截图	3
2 OOP 设计详解.....	4
2.1 类继承图	4
2.2 抽象基类 GameObject	4
2.3 派生类职责	4
2.4 Game 与 Map: 组合优于继承	5
2.5 虚函数如何体现多态	5
2.6 dynamic_cast 的局部使用	5
2.7 如果不使用 OOP	5
3 现代 C++ 特性运用分析.....	6
3.1 vector 管理动态集合	6
3.2 范围 for 遍历	6
3.3 unique_ptr 与 make_unique (重点)	7
3.4 其他现代风格	7
4 核心难点与解决方案.....	8
4.1 难点描述	8
4.2 定位思路	8
4.3 解决方案	8
4.4 结果与可改进点	8
5 编译与运行说明	9
5.1 Visual Studio 运行	9
5.2 命令行构建示例	9
6 总结与心得	9

1 项目概述

功能、玩法规则与运行效果

EasyBomber 是一个使用 C++17 和 EasyX 图形库实现的单人炸弹人小游戏。玩家在 15×21 的网格地图中移动，通过放置定时炸弹清除可破坏砖块并消灭全部怪物。项目采用 Visual Studio C++ 桌面工程组织，地图、玩家、怪物、炸弹和爆炸效果分别由独立类负责。

操作规则：方向键控制玩家在道路格上移动；Space 键在当前位置放置炸弹；R 键重新开始；Esc 键退出。炸弹引信为 2 秒，爆炸从中心向上、下、左、右各传播 2 格；固定墙体会立即阻断火焰，可破坏砖块会被炸毁并阻断其后的传播。玩家接触怪物或被火焰命中则失败，消灭全部怪物则胜利。

项目特点 固定时间步保护、双缓冲绘制、基于网格的碰撞检测、动态对象统一管理，以及通过虚函数实现的更新/绘制多态。

1.1 运行截图

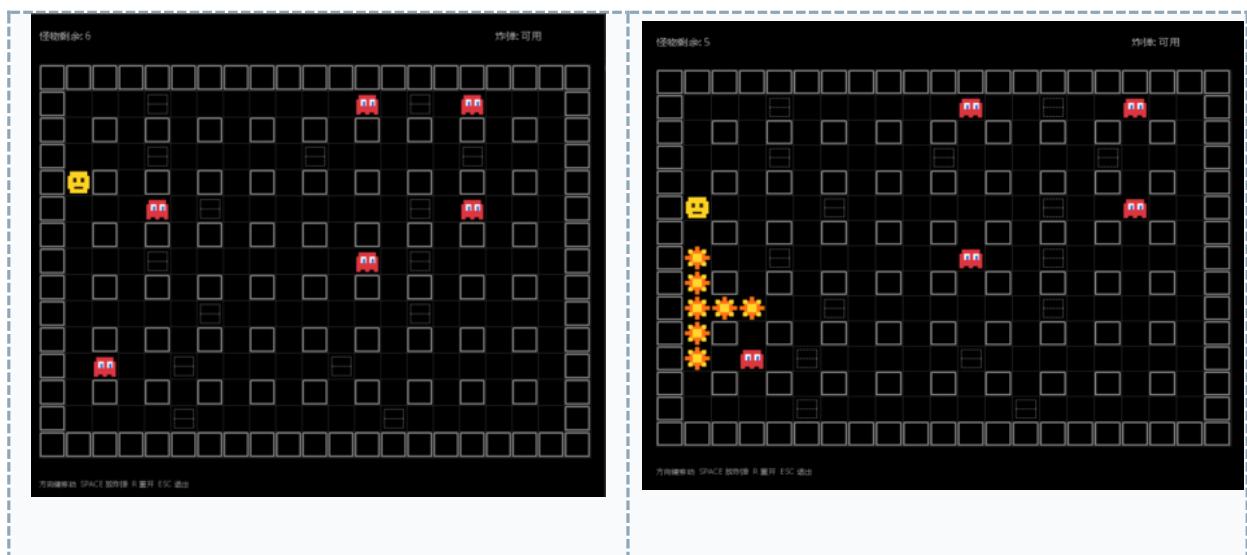
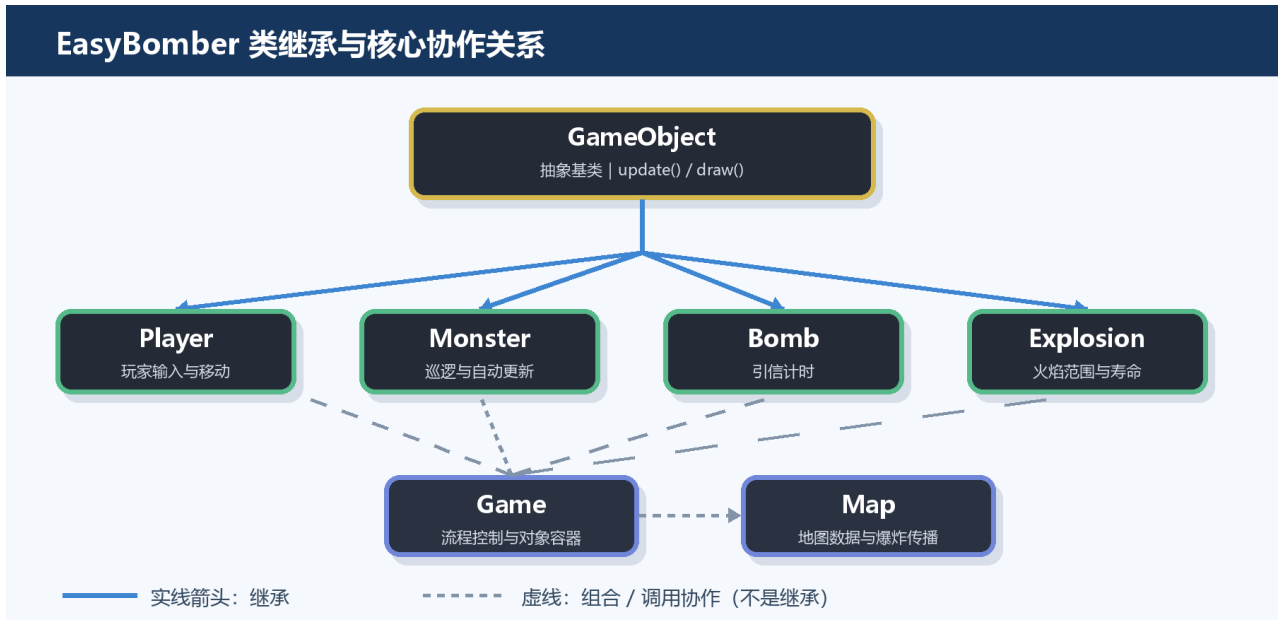


图 1 EasyBomber 运行效果（请插入两张本人实际运行截图）

2 OOP 设计详解

重点：继承关系、职责划分与运行时多态

2.1 类继承图



2.2 抽象基类 GameObject

GameObject 抽取了所有动态游戏对象共有的网格位置 `pos`、活动标记 `active`，以及 `update(int dt, const Map&)` 和 `draw()` 两个行为接口。两个接口均为纯虚函数，因此基类不能被直接实例化，派生类必须给出自己的更新和绘制方式。虚析构函数保证通过基类指针销毁派生对象时能够正确执行完整析构过程。

`active` 与 `remove()` 采用“标记删除”策略：碰撞检测阶段只把对象标记为失效，统一清理阶段再从容器移除。这样可避免在遍历 `vector` 时立即删除对象造成迭代位置变化或悬空指针。

2.3 派生类职责

类	核心状态	主要职责
Player	继承 <code>pos</code> 、 <code>active</code>	响应方向键；检查地图与炸弹占位；绘制玩家贴图
Monster	<code>route</code> 、 <code>routeId</code> 、 <code>moveTime</code>	按固定路径巡逻；按 450 ms 节拍移动；遇阻等待
Bomb	<code>timer</code> 、 <code>FUSE_TIME</code>	累计引信时间；提供倒计时显示；到时等待 Game 引爆
Explosion	<code>cells</code> 、 <code>timer</code>	保存爆炸波及格；判断命中；450 ms 后自动失效

2 OOP 设计详解（续）

Game、Map 的协作与多态调用

2.4 Game 与 Map：组合优于继承

Game 不是某一种游戏对象，而是流程编排者，因此没有继承 `GameObject`。它组合一个 `Map`、一个 `unique_ptr<Player>`，以及一个 `vector<unique_ptr<GameObject>>` 动态对象集合，按“输入—更新—碰撞—清理—绘制”的顺序推进每一帧。`Map` 同样不是动态实体，它负责二维地图数据、出生点、道路判定、砖块破坏和爆炸传播计算。

这一结构体现了“is-a 用继承，has-a 用组合”的原则：`Monster` 是一种 `GameObject`，所以采用继承；`Game` 拥有地图和对象，所以采用组合。职责边界清楚后，地图规则的修改不会侵入贴图绘制，怪物移动逻辑的调整也不会破坏主循环。

2.5 虚函数如何体现多态

`objects` 容器中的静态类型统一为 `unique_ptr<GameObject>`，但其中实际保存 `Monster`、`Bomb` 和 `Explosion`。`Game` 在更新和绘制时只调用 `objects[i]->update(...)` 与 `objects[i]->draw()`，运行时会根据对象的真实类型选择对应的重写函数：怪物移动、炸弹计时、爆炸消失，以及三类对象各自的绘制逻辑。主循环不需要用大量 `if/switch` 判断对象种类，这正是运行时多态。

设计价值 新增一种动态对象时，只需继承 `GameObject` 并实现 `update/draw`，再放入 `objects` 容器；`Game` 的通用更新和绘制循环无需改写。

2.6 dynamic_cast 的局部使用

通用行为通过虚函数处理；只有“仅属于某一种类型”的业务操作才使用 `dynamic_cast`。例如引爆阶段需要调用 `Bomb::isReady()`，碰撞阶段需要调用 `Explosion::hits()`，统计阶段需要识别 `Monster`。这样的使用范围较小，避免把所有逻辑都退化为类型判断。若后续扩展，可进一步使用对象类别枚举、事件系统或双分派减少类型转换。

2.7 如果不使用 OOP

比较项	过程式写法	当前 OOP 写法
数据组织	多组全局数组分别保存位置、计时器、贴图和状态	状态封装在 <code>Player</code> 、 <code>Monster</code> 、 <code>Bomb</code> 、 <code>Explosion</code> 内
更新/绘制	主循环使用大量类型标志与 <code>if/switch</code> 分支	基类指针统一调用虚函数，自动分派
扩展维护	新增对象需修改多个数组、分支和清理代码	新增派生类并接入容器，改动范围较集中

因此，OOP 带来的核心收益不是“类更多”，而是变化被限制在职责明确的模块中。可维护性体现在定位问题更直接，可扩展性体现在新功能不必反复改动主循环，资源安全则由 `RAII` 和智能指针进一步保障。

3 现代 C++ 特性运用分析

从安全性、可读性与简洁性比较现代写法

3.1 vector 管理动态集合

项目使用 vector 保存怪物路线、爆炸覆盖格、地图二维数据和动态对象集合。与固定长度 C 数组相比，vector 能根据 push_back 自动扩容，并通过 size() 提供当前元素数量，适合怪物死亡、炸弹生成和爆炸消失这类运行期变化。传统写法需要预估最大容量、维护计数变量，并在插入删除时手工移动元素，容易越界或出现计数不同步。

代码片段 1 | 动态对象集合

```
unique_ptr<Player> player;           // 单独持有，方便高频访问
vector<unique_ptr<GameObject>> objects; // Monster/Bomb/Explosion 混在一起，多态管理
```

3.2 范围 for 遍历

Explosion::draw() 使用范围 for 逐个取得 cells 中的 GridPosition。代码直接表达“对每个爆炸格绘制贴图”，不需要管理下标，也不会误写循环边界。传统下标循环必须写初始化、条件和自增，并反复使用 cells[i]，信息噪声更大。若元素无需修改，还可进一步写成 const GridPosition&，减少复制并表达只读意图。

代码片段 2 | 遍历爆炸覆盖格

```
// 在波及的每一个格子上画火焰贴图（其他实体只画一格，这个画一片）
void Explosion::draw() const {
    static IMAGE image;
    static bool loaded = false;
    if (!loaded) {
        loadimage(&image, L"assets\\explosion.bmp", Map::BLOCK, Map::BLOCK);
        loaded = true;
    }
    for (GridPosition cell : cells) {
        putimage(Map::getX(cell.col), Map::getY(cell.row), &image);
    }
}
```

3 现代 C++ 特性运用分析（续）

智能指针、RAII 与对象创建

3.3 unique_ptr 与 make_unique（重点）

自学特性 | 智能指针 本项目使用 `unique_ptr` 表达对象的唯一所有权，并使用 `make_unique` 创建 `Player`、`Monster`、`Bomb` 和 `Explosion`。

如果使用传统裸指针，Game 需要在 `reset()`、正常退出、对象死亡和异常路径中手动 `delete`；任何遗漏都可能造成内存泄漏，重复释放又可能导致程序崩溃。`unique_ptr` 在离开作用域或从 `vector` 中擦除时自动析构对象，资源生命周期与容器生命周期绑定，符合 RAII 思想。

`make_unique` 还把“分配内存”和“交给智能指针管理”合并成一个表达式，避免 `new` 与所有权接管之间出现异常窗口。`objects.push_back(make_unique<Monster>(...))` 同时清楚表达：创建一个 `Monster`，并把它的所有权转移给 `objects`。

代码片段 3 | 智能指针创建与所有权转移

```
void Game::reset() {
    map.reset(); // 地图从字符重建
    state = PLAYING;
    player = make_unique<Player>(map.getPlayerStart());
    objects.clear();

    vector<GridPosition> starts = map.getMonsterStarts();
    for (int i = 0; i < starts.size(); i++) {
        objects.push_back(make_unique<Monster>(starts[i], monsterRoutes[i]));
    }
}

for (int i = 0; i < objects.size(); i++) {
    Bomb* bomb = dynamic_cast<Bomb*>(objects[i].get()); // 筛选出 Bomb
    if (bomb != nullptr && bomb->isActive() && bomb->isReady()) {
        center = bomb->getPos();
        cells = map.getBlast(center, 2); // 算爆炸范围
        bomb->remove();
        exploded = true;
    }
}
if (!exploded) return;
```

3.4 其他现代风格

`GridPosition` 使用聚合初始化 `GridPosition p{row, col}`，语义直接且避免字段遗漏；派生类方法使用 `override`，编译器可以检查函数签名是否真正重写基类虚函数；`GameObject` 的析构函数写成 `virtual ~GameObject() = default`，既保证多态析构，又避免无意义的手写空函数。这些写法共同减少样板代码，并把设计意图交给编译器校验。

需要说明的是，项目部分循环仍采用 `int` 下标与显式类型，这是为了保持课堂代码易读。若继续改进，可在不涉及删除的遍历中使用范围 `for` 或 `auto`，在索引场景中改用 `std::size_t`，并用 `erase-remove_if` 简化失效对象清理。

4 核心难点与解决方案

动态对象生命周期、遍历删除与同帧碰撞

4.1 难点描述

最棘手的设计问题是：炸弹到时需要消失并立即生成 Explosion；爆炸又可能在同一帧杀死怪物或玩家；随后这些失效对象还要从 vector 中删除。如果在遍历 objects 时直接 erase，后续元素会前移，循环下标可能跳过对象；如果保存了被删除对象的裸指针，还可能形成悬空指针。

4.2 定位思路

问题可以通过观察“连续对象未被清理”“爆炸偶尔漏判”或调试器中的容器大小变化定位。重点检查三个时间点：对象何时被标记失效、Explosion 何时加入容器、clearObjects() 何时真正擦除。把每一帧拆成明确阶段后，可以发现碰撞检测与删除操作混在一起是风险来源。

4.3 解决方案

① 延迟删除	remove() 只把 active 设为 false，不在碰撞或引爆遍历中立即 erase。
② 分阶段更新	先检查上一帧碰撞，再更新对象；引爆后再次检查碰撞，使新火焰同帧生效。
③ 统一清理	clearObjects() 最后执行；删除元素后下标不递增，避免跳过后移元素。
④ RAII 托管	vector<unique_ptr<GameObject>> 在 erase 时自动析构对象，不需要手工 delete。

4.4 结果与可改进点

采用上述流程后，对象状态变化具有确定顺序：本帧产生的爆炸能够立即参与伤害判定，失效对象不会继续 update/draw，容器清理也不会遗漏元素。进一步改进时，可使用 std::erase_if (C++20) 或 erase-remove_if (C++11/17) 统一删除逻辑，并为“炸弹—爆炸—碰撞—清理”编写无图形界面的单元测试。

代码片段 4 | 可选：延迟删除与统一清理

```
void remove() { active = false; } // 标记待删除，不真的 delete，由 Game 统一清理

// erase 后元素会前移，索引不能直接 ++，所以用 while
void Game::clearObjects() {
    int i = 0;
    while (i < objects.size()) {
        if (!objects[i]->isActive()) objects.erase(objects.begin() + i); // 删了 i 不变
        else i++;
    }
}
```

5 编译与运行说明

环境、依赖与构建命令

操作系统	Windows 10/11 (x64)
开发环境	Visual Studio 2026, 安装“使用 C++ 的桌面开发”工作负载
编译器/工具集	MSVC, Platform Toolset v145
语言标准	C++17 (满足 C++11 或更高版本要求)
图形依赖	EasyX; 链接 EasyXw.lib, 运行目录需包含 assets 文件夹

5.1 Visual Studio 运行

安装 EasyX 并完成 Visual Studio 配置后, 双击 EasyBomber.sln, 选择 Debug | x64 或 Release | x64。按 F5 启动调试, 或按 Ctrl+F5 直接运行。工程的生成后事件会把 assets 目录复制到 bin\Debug 或 bin\Release, 避免运行时找不到贴图。

5.2 命令行构建示例

```
msbuild EasyBomber.sln /p:Configuration=Release /p:Platform=x64
.\bin\Release\EasyBomber.exe
```

命令应在“Developer PowerShell for Visual Studio”中、项目根目录下执行。若链接阶段提示找不到 EasyXw.lib, 应检查 EasyX 是否安装到当前 MSVC 工具集, 并核对项目的附加库目录。

6 总结与心得

通过 EasyBomber 项目, 我把继承、虚函数、多态、组合和资源管理落实到了一个可运行的小游戏中。最直接的收获是理解了“共同接口与不同实现”的价值: GameObject 统一 update/draw 后, 主循环能够专注于流程, 不必知道每个对象的全部细节。

项目仍有不足: 怪物只沿固定路线巡逻, 关卡数据写在源码中, dynamic_cast 使用较多, 贴图加载方式也可以抽成资源管理器。后续可增加道具、连锁爆炸、多关卡配置和简单寻路, 并把碰撞与地图规则拆成可单元测试的纯逻辑模块。总体而言, 本项目让我体会到, OOP 的重点并非堆叠类, 而是让职责、所有权和变化边界变得清楚。